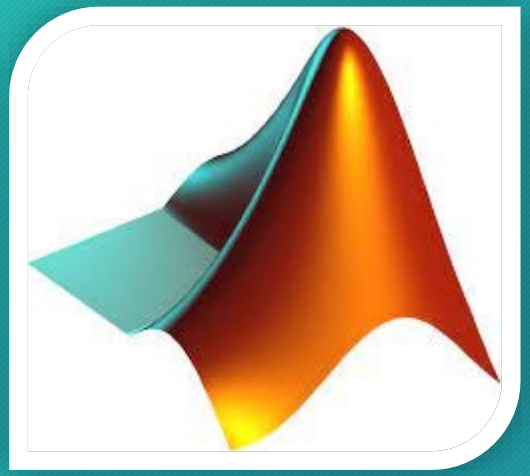


Communication Design

LAB.5 Wireless Communications

Simulate a Basic Digital Communications Link

Ass.Lec.Ammar Alabbassi

Course Outline:

✓1- Simulate a Basic Digital Communications Link

- Implement the essential components of a single-carrier digital communications system, including additive white Gaussian noise.

2- Pulse-Shaping Filters

- Incorporate transmit and receive filters into your simulation.

3- Multipath Channels

- Model a multipath channel and evaluate the impact on link quality.

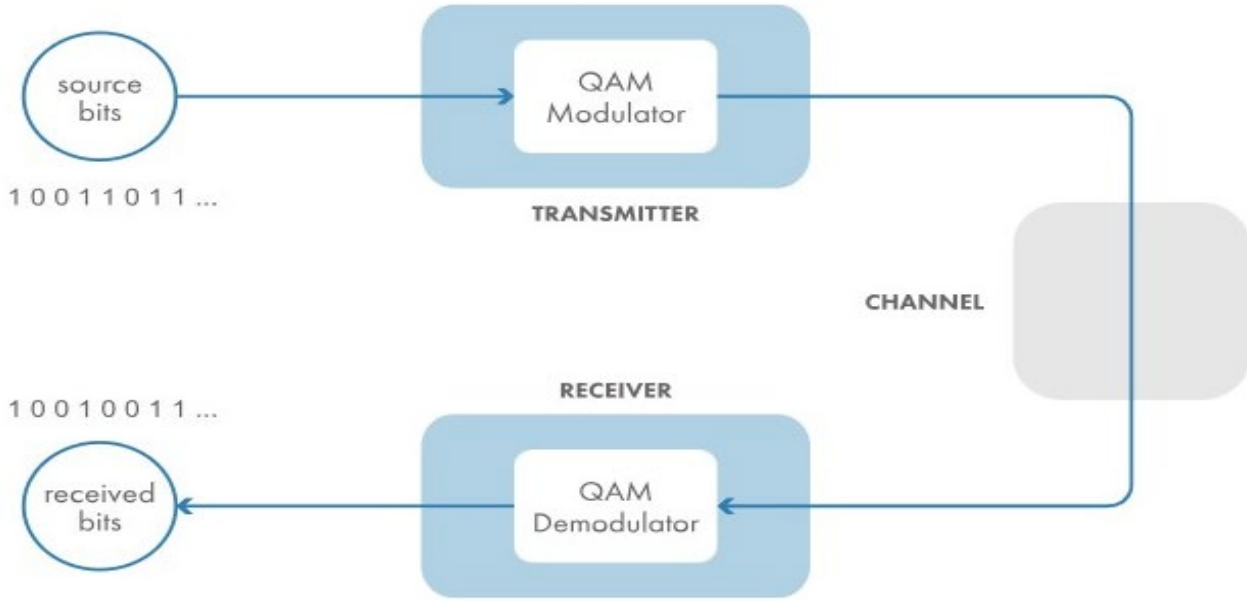
4- OFDM

- Implement orthogonal frequency-division multiplexing (OFDM), a multicarrier modulation scheme.

1- Simulate a Basic Digital Communications Link

Back-to-Back QAM Link

In this activity, you'll simulate a simple back-to-back communications link that uses quadrature amplitude modulation, or QAM. You'll generate a source signal of random bits, modulate the source bits using QAM, demodulate the QAM symbols back into bits, and compare the demodulated bits to the source bits to see if they match.



1- Simulate a Basic Digital Communications Link

Modulation and Demodulation

Task 1

Background

- The first step in any digital communications simulation is to create a source signal. Here, the source signal is a sequence of bits—or, in MATLAB terms, a vector of 0s and 1s.
- You can use the randi function to create a column vector of randomly-generated 0s and 1s.

x = randi([0,1],numBits,1)

- The first input sets the range of integers (0 and 1), the second input is the number of rows, and the third input is the number of columns.
- The output x is a column vector containing **numBits** bits.

TASK

Create a column vector named **srcBits** containing **20,000** randomly-generated bits.

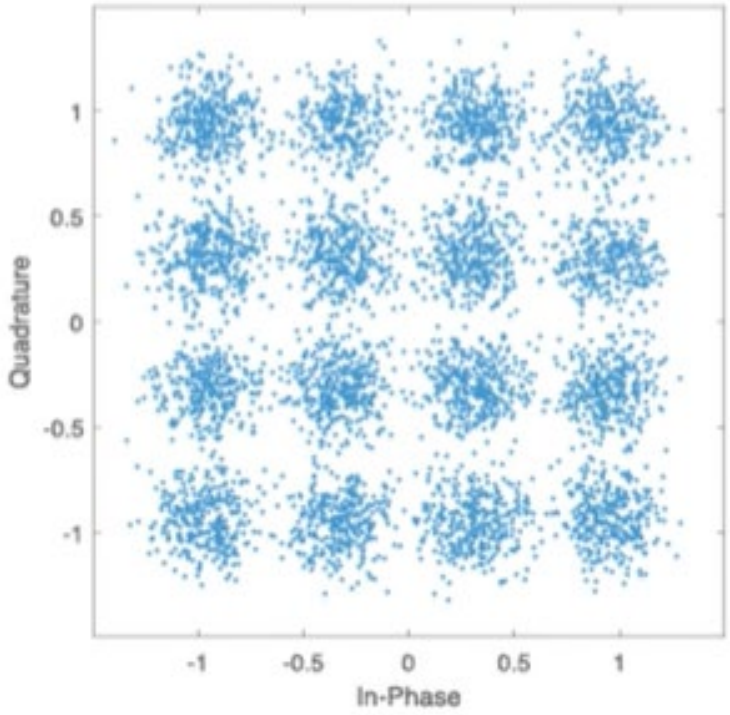
```
numBits = 20000;
srcBits = randi([0,1],numBits,1);
```

srcBits = *20000×1*

	1
1	0
2	0
3	0
4	0
5	1
6	1
7	0
8	0

Task 2

- 16-QAM is a common single-carrier modulation scheme. It maps 4 input bits to one of 16 complex numbers, called symbols. For each complex-valued symbol, the real and imaginary parts represent the in-phase and quadrature components, respectively, of a waveform.



- The "16" in 16-QAM is the modulation order.
- It's useful to store the modulation order in a variable so that it can be used throughout your simulation.

TASK

Create a variable named modOrder and set its value to 16.

```
modOrder = 16
```

Task 3

- The **qammod** function creates a **modulated QAM signal** from an input sequence.

  y = qammod(x,modOrder)

- The modulated signal **y** is a sequence of **QAM symbols**—or, **in MATLAB terms, a vector of complex numbers**.

- When the input sequence is made up of bits, set **"InputType"** to "bit".

  y = qammod(x,modOrder,"InputType","bit")

modOut = *5000×1 complex*

	1
1	3.0000 + 3.0000i
2	-3.0000 + 1.0000i
3	-3.0000 + 1.0000i
4	3.0000 - 1.0000i
5	-1.0000 - 3.0000i
6	3.0000 + 1.0000i
7	3.0000 + 1.0000i
8	1.0000 + 3.0000i

TASK

Create a 16-QAM signal from the bit sequence srcBits. Name the modulated signal modOut.

modOut = qammod(srcBits,modOrder,"InputType","bit")

Task 4

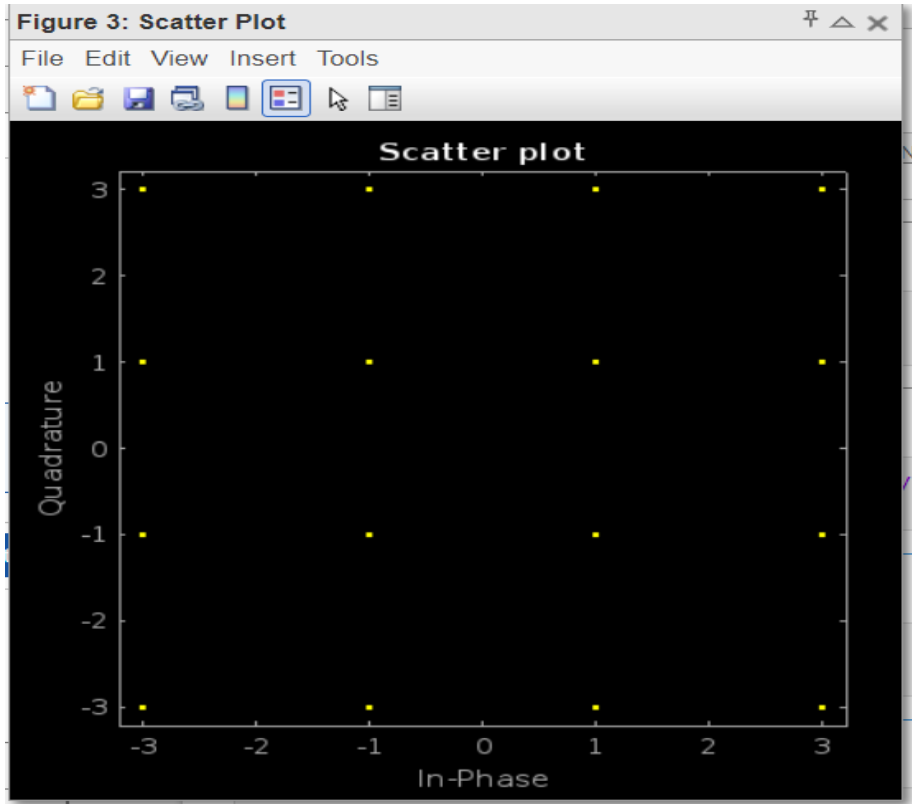
- You can look at the constellation of the **modulated QAM** signal by using the **scatterplot function**. This function plots the imaginary part **(quadrature)** vs. the real part **(in-phase)** of each complex-valued QAM symbol.

```
scatterplot(y)
```

TASK

Create a scatter plot of the modulated signal modOut.

```
scatterplot(modOut)
```



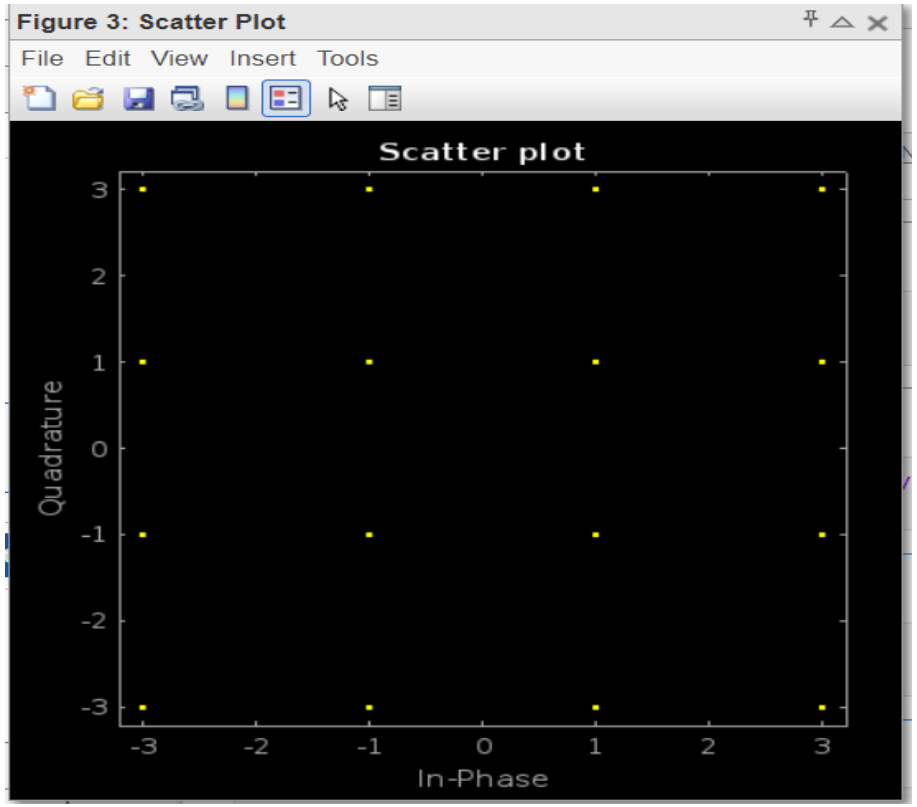
Task 5

- What you see in the scatter plot is the **ideal** 16-QAM constellation—16 complex-valued symbols arranged in a square, with each symbol **equally spaced** from its neighbors.
- The greater the separation between points, the better the error rate performance. Later, when you add noise to the simulation, you'll see the received observations depart from this ideal configuration.
- For now, the simulation assumes that the modulated signal is unchanged when it's received by the receiver.

TASK

Create a variable named chanOut and set it equal to the modulated 16-QAM signal modOut.

```
chanOut = modOut
```



Task 6

- In the receiver, demodulation turns the 16-QAM symbols back into bits.
- Use the qamdemod function to demodulate the received signal. It has almost the same syntax as qammod. The only difference is that qamdemod specifies the "OutputType".

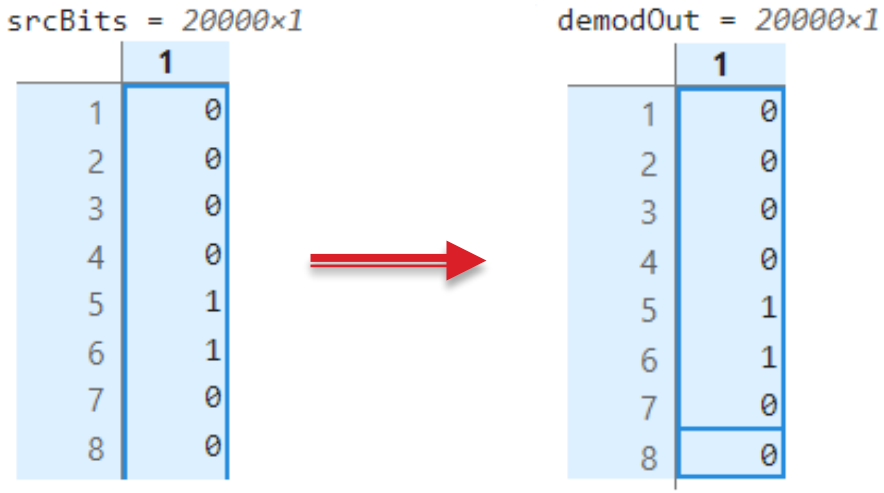
```
z = qamdemod(y,modOrder,"OutputType","bit")
```

- The output z is a column vector of 1s and 0s—the received bits.

TASK

Demodulate the received signal chanOut. Name the output demodOut.

```
demodOut = qamdemod(chanOut,modOrder,"OutputType","bit")
```



Task 7

- Since the 16-QAM signal was not subject to any **channel effects**, the received bits should match the source bits exactly.
- To see if the two vectors are identical, use the **isequal function**.

  check = isequal(x1,x2)
- The output check is a logical variable with **value** 1 if the two vectors are identical and 0 if they are not.

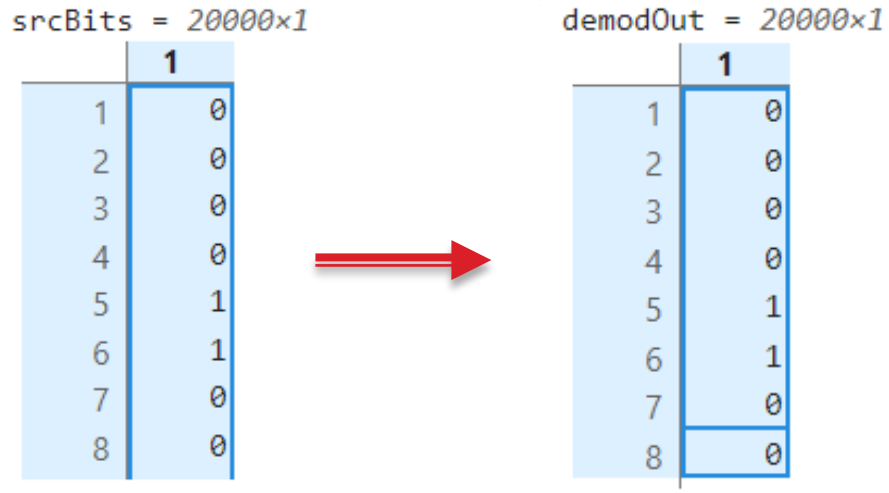
TASK

Check if the source bits srcBits and received bits demodOut are identical. Name the output check.

To view the result, leave off the semicolon at the end of the command.

```
check = isequal(srcBits,demodOut)

check = logical
    1
```



1- Simulate a Basic Digital Communications Link

Add Noise

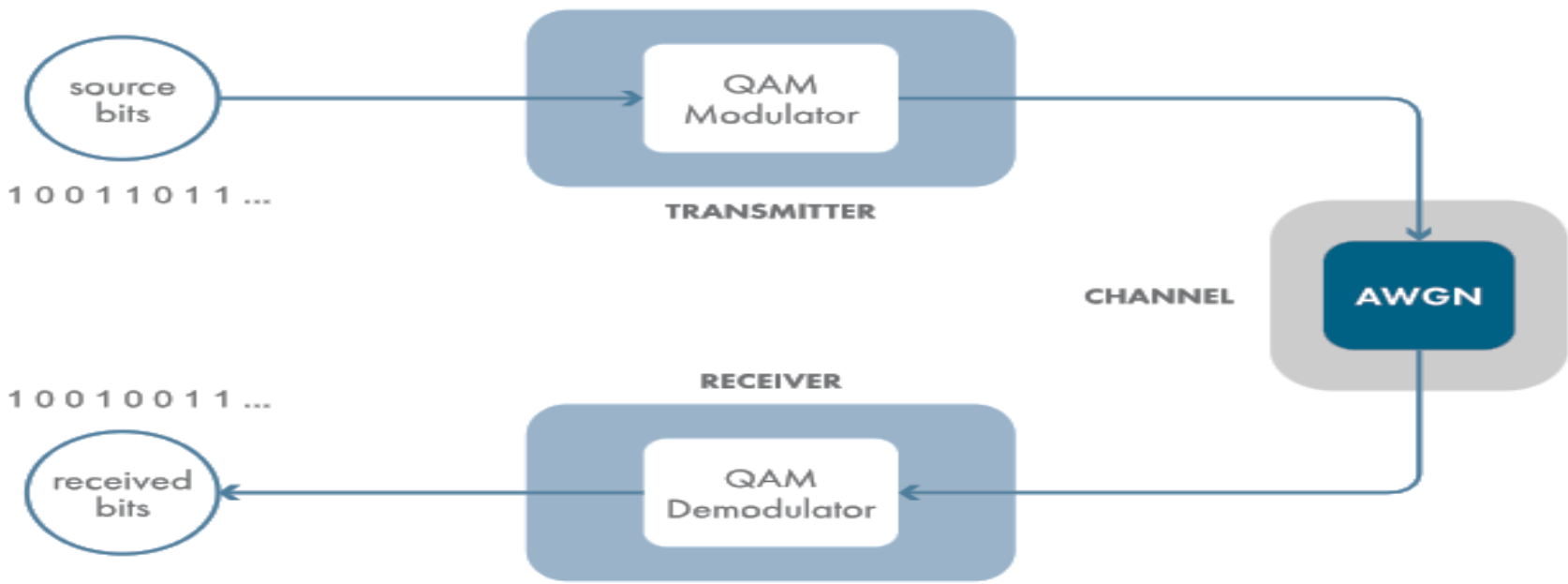
1- Simulate a Basic Digital Communications Link

Add Noise

A common corruption, or noise source, in the received signal is due to electron motion from the electronics in the receiver. This noise is modeled as part of the channel.

The noise is called *additive white Gaussian noise*, or AWGN, because it's typically *added* to the signal, has a flat (or *white*) spectral density, and has a *Gaussian* probability density function.

In this activity, you'll add noise to your 16-QAM link simulation and note its effects.



Task 1

Background

- The awgn function applies additive white Gaussian noise (AWGN) to its input signal. The amount of noise added is specified as a signal-to-noise ratio, assuming the input signal has an average power of 1.

- Fortunately, you can tell the qammod function to output a signal with an average power of 1. Simply set "UnitAveragePower" to true.

```
sigOut = qammod(bits,modOrder,..."InputType","bit",... "UnitAveragePower",true)
```

- **Pro tip**: Have you ever had trouble reading a long line of code? You can use ... to split a long command into multiple lines.

TASK

- **Create a 16-QAM signal from the bit sequence srcBits. Specify the output signal to have unit average power.**
- **Name the output signal modOut.**

```
modOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);
```

Task 2

- Next, model the noisy channel by applying AWGN to the modulated signal.
- You can use the **awgn** function to apply **AWGN** to a signal. The noise power added is specified as a signal-to-noise ratio, SNR, measured in decibels (dB).

```
SNR = 7  % dB
sigOut = awgn(sig,SNR)
```

TASK

- Define a variable named SNR with the value 15.
- Apply AWGN to the modulated signal modOut with signal-to-noise ratio SNR. Name the output signal chanOut.

```
SNR = 15;    % dB
chanOut = awgn(modOut,SNR);
```

Task 3

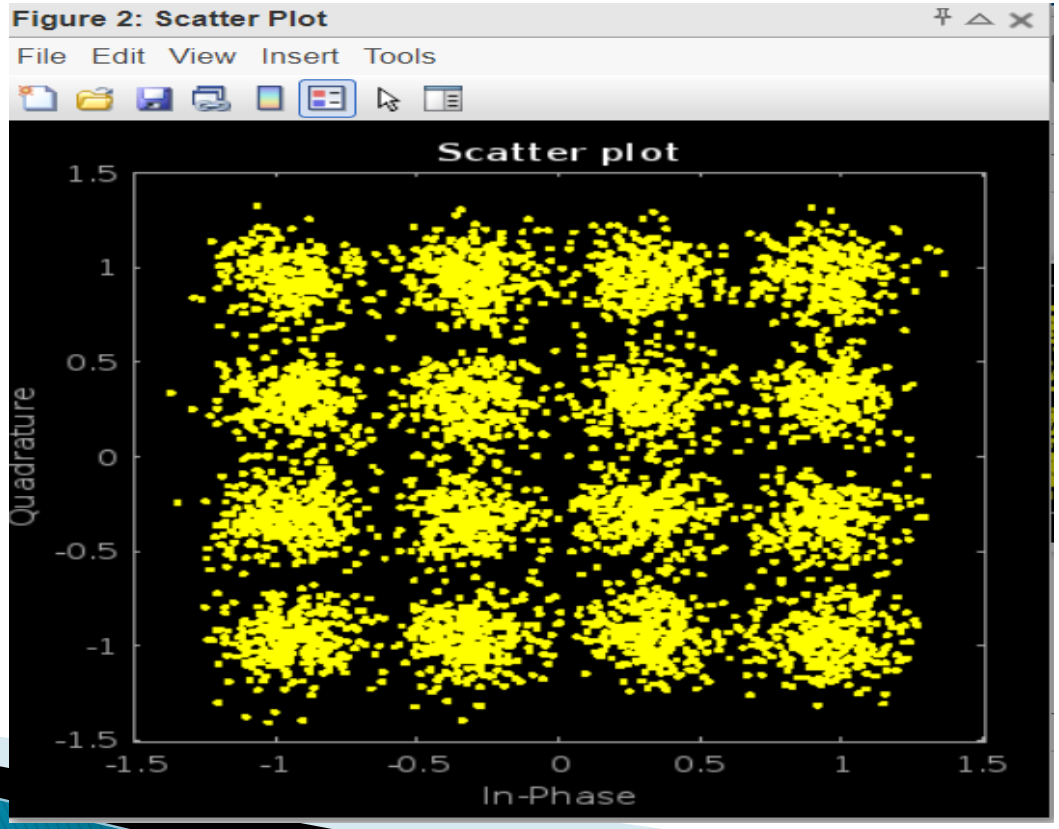
- To see the impact of the noise, you can view the constellation of the channel output using the scatterplot function, as before.

scatterplot(sig)

TASK

Create a scatter plot of the channel output chanOut.

scatterplot(chanOut)



Task 4

- Now you can see that the noise has perturbed the symbols from their ideal values.
- The **qamdemod** function uses a nearest-neighbor decision rule to map received observations back to the nearest ideal symbol constellation point. If an observation has been perturbed too much, it will get mapped to the wrong point. This results in symbol errors, in turn yielding bit errors.

- Since you set "UnitAveragePower" when you called qammod, you must also set it in qamdemod.

```
sigOut = qamdemod( sig,modOrder,... "OutputType","bit",... "UnitAveragePower",true)
```

TASK

Demodulate the signal **chanOut**, and name the output **demodOut**. Specify the signal has unit average power.

Check if the source bits **srcBits** and received bits **demodOut** are identical using the **isequal** function. Name the result check.To view the result, leave off the semicolon at the end of the command.

```
demodOut = qamdemod(chanOut,modOrder,"OutputType","bit","UnitAveragePower",true);
check = isequal(srcBits,demodOut)
```

```
Check = logical
 0
```

Task 4

- Now you can see that the noise has perturbed the symbols from their ideal values.
- The **qamdemod** function uses a nearest-neighbor decision rule to map received observations back to the nearest ideal symbol constellation point. If an observation has been perturbed too much, it will get mapped to the wrong point. This results in symbol errors, in turn yielding bit errors.

- Since you set "UnitAveragePower" when you called qammod, you must also set it in qamdemod.

```
sigOut = qamdemod( sig,modOrder,... "OutputType","bit",... "UnitAveragePower",true)
```

TASK

Demodulate the signal **chanOut**, and name the output **demodOut**. Specify the signal has unit average power.

Check if the source bits **srcBits** and received bits **demodOut** are identical using the **isequal** function. Name the result check.To view the result, leave off the semicolon at the end of the command.

```
demodOut = qamdemod(chanOut,modOrder,"OutputType","bit","UnitAveragePower",true);
check = isequal(srcBits,demodOut)
```

```
Check = logical
 0
```

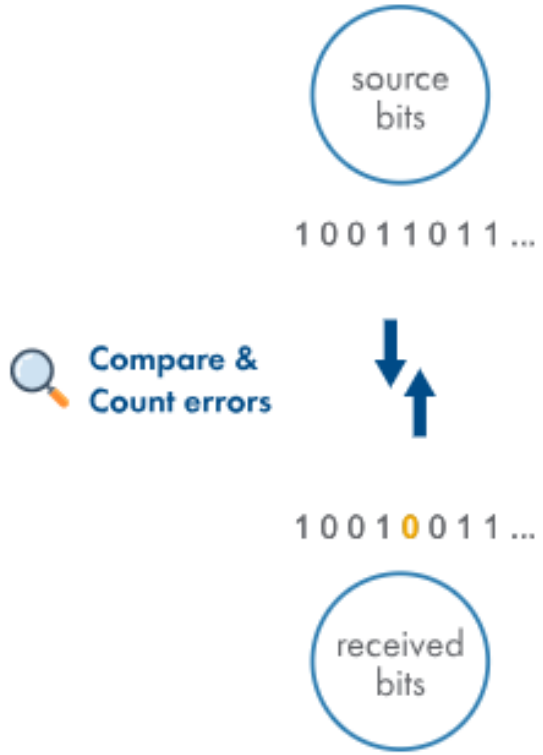
1- Simulate a Basic Digital Communications Link

Calculate the Bit Error Rate

1- Simulate a Basic Digital Communications Link

Calculate the Bit Error Rate

Noise can introduce errors in the received bits. Comparing each bit sent to the corresponding bit received tells you if there are any errors. The fraction of bits that contain bit errors is a key way to measure the performance of a communications system.



Task 1

This code simulates a 16-QAM link with AWGN.

```
% Simulation parameters
numBits = 20000;
modOrder = 16;
% Create source signal and apply 16-QAM modulation
srcBits = randi([0,1],numBits,1);
modOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);



% Apply AWGN
SNR = 15; % dB
chanOut = awgn(modOut,SNR);
scatterplot(chanOut)
% Demodulate received signal
demodOut = qamdemod(chanOut,modOrder,"OutputType","bit","UnitAveragePower",true);
```

Task 1

Background

- To count bit errors, you start by performing a bit-by-bit comparison of the source bits and the received bits.
- The logical operator ~= (not equal) compares two arrays element-by-element.

  isErr = x~=z
- The output isErr is a logical array. It has the value 1 where the elements of x and z are not equal and 0 where they are.

TASK

Compare the source bits **srcBits** and received bits **demodOut** element-by-element to identify bit errors. Name the result **isBitError**.

```
isBitError = srcBits~=demodOut;
```

Task 2

- You can count the number of bit errors using the nnz function. It counts the number of nonzero elements in an array.

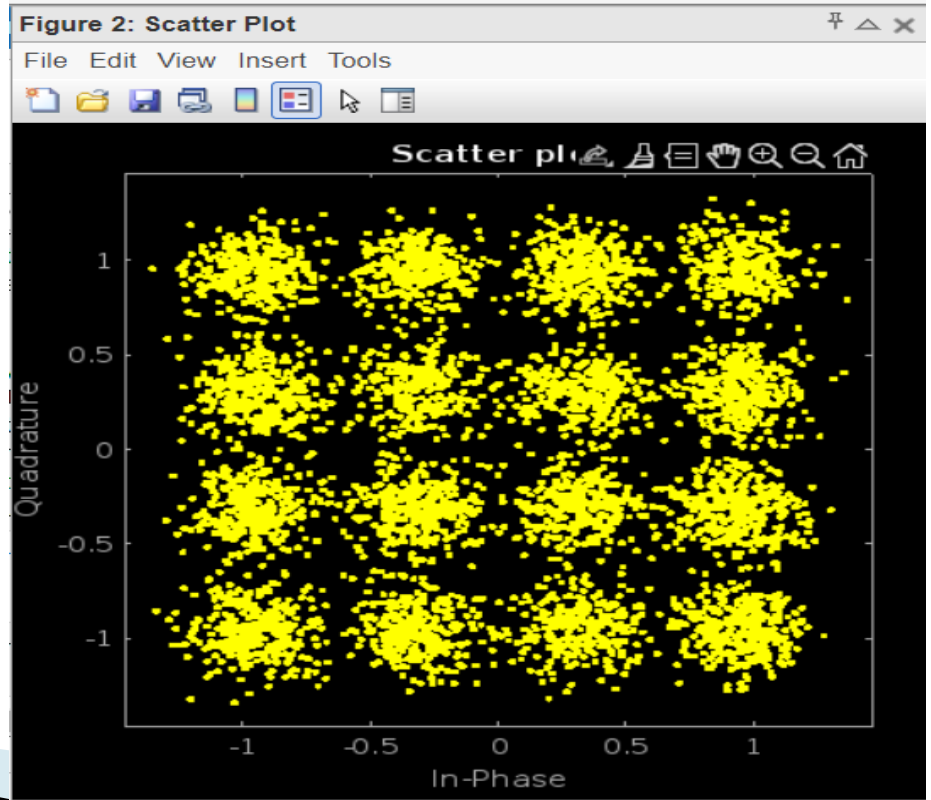
numErr = nnz(isErr)

TASK

Count the number of bit errors, and store it in a variable named **numBitErrors**.

numBitErrors = nnz(isBitError)

numBitErrors = 84



Task 3

- The bit error rate **(BER)** is calculated by dividing the number of bit errors by the total number of bits.

TASK

- Calculate the bit error rate, and store it in a variable named BER.
- The number of bits is stored in the variable numBits.

BER = numBitErrors/numBits

BER = 84/20000

BER = 0.0042